MachBoost: Speculative Decoding from Local Context for On-Device LLMs

Vistrit Pandey
vistrit@blinkfire.com

July 2026

Abstract

Autoregressive language model inference is slow in a very ordinary way: for most decoders, one generated token still means one more target-model step. Speculative decoding can reduce that serial work, but many practical variants require a draft model, modified model heads, or serving infrastructure that is awkward for local use. This paper explores a smaller question: when a local task is already grounded in nearby text, can that text itself draft useful continuations? MachBoost indexes prompt and local-context tokens, proposes n-gram continuations, and verifies them with the target model before committing any generated token. On an Apple M1 Max with 32 GB memory, the initial MLX suite reaches a 1.38× median speedup across 35 runs with token-for-token agreement against serial greedy decoding. A stricter 64-token MLX suite, repeated three times, reaches a 3.00× aggregate median speedup over 90 rows. A direct Hugging Face comparison on `Qwen/Qwen2.5-3B-Instruct` is more mixed but still useful: built-in prompt lookup reaches 2.11× median speedup, while MachBoost context mode reaches 2.47× over the same six fixtures. These results are not a universal acceleration claim. They show a package-shaped path for local, grounded workloads where the next tokens are often already nearby.

1 Introduction

Running LLMs on a laptop has become realistic, but it has not made autoregressive decoding disappear. A small model on Apple Silicon may fit in memory and still feel slow because the loop is serial: produce a token, update the prefix, run the model again. That bottleneck is especially visible in developer workflows, where the model often sits inside an editor, command runner, notebook, or local assistant.

Speculative decoding offers a way around part of this loop. A cheap source guesses several future tokens; the target model checks the guess; accepted tokens are emitted without paying for one full target step each. Prior work usually gets the draft from another model, extra heads, or a reference sequence. MachBoost takes the reference idea and applies it to the local machine. If the user is asking about a repo, config, note, policy, or retrieved passage, the likely continuation may already be present in that local text.

The scope is deliberately modest. MachBoost does not try to accelerate every prompt. It tries to accelerate grounded prompts without changing the target model's greedy output. If the local text is unhelpful, the method can be slower; the implementation exposes that fact through measurement and falls back to the serial path.

This paper makes five concrete contributions:

- a local-context drafter that proposes continuations from prompt text, files, and retrieved material;
- verifier-backed generation for MLX and Hugging Face-style runtimes, including a fix for a cache-path divergence observed in MLX;
- a Python API that measures a workload before deciding whether to use the boosted path;
- Apple Silicon evaluations with exact token agreement across the reported MLX and Hugging Face comparison rows;
- a direct comparison against Hugging Face's built-in prompt lookup decoding on a 3B local model.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [5]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [3]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [2].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [10]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [6]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [7]. Hugging Face Transformers exposes prompt lookup through `prompt_lookup_num_tokens` in assisted generation [4]. MachBoost differs mainly in packaging and operating assumptions: it treats local files and retrieved snippets as ordinary inputs to the accelerator, keeps benchmark data machine-readable, and makes low-overlap prompts a measured fallback case rather than an error.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [8]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Our measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [9].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. MachBoost receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning the same sequence y that greedy decoding with M would return.

This paper stays with greedy decoding. Sampling-compatible speculative decoding is possible in principle, but greedy argmax verification gives a simple correctness check: the accelerated sequence must match the serial baseline token-for-token.

4 Method

The implementation has three moving parts: a drafter, a verifier, and a decision rule for whether to use the drafter on a workload.

4.1 Drafting from Nearby Text

The drafter indexes token n-grams from a corpus C. At generation time it looks at the current prefix $p = x \| \hat{y}$, searches for suffixes of p that occur in C, and proposes the tokens that followed the best matching span. Longer suffix matches are preferred, with draft length as a secondary signal. The design is intentionally plain: no learned draft model, no training step, and no hidden state from the target model.

4.2 Verification Against the Target Model

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens are equal to the target model's greedy continuation. For verifier-capable runtimes, this can be done by scoring $p \| d$ in a single target call and comparing target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. If a residual target token is available at the first mismatch, it is emitted to preserve progress. Otherwise the system falls back to a normal target step.

The invariant is:

$$\text{Boost}(M, x, C, T) = \text{Greedy}(M, x, T)$$

for a maximum generation length T. In the implementation, every benchmark row records an `output_match` flag by comparing boosted and baseline token IDs directly.

4.3 Keeping the Cache Path Stable

One engineering detail turned out to matter more than expected. In MLX, extending a prompt cache token-by-token is not always numerically identical to rebuilding a long prefix in a single call. During testing, a non-trimmable cache could partially accept a draft, rebuild after rejection, and later choose a different argmax than the serial cache path.

MachBoost handles this by verifying drafts on a cloned trial cache when the runtime supports cloning. Fully accepted drafts commit the trial cache. On partial rejection, if the trial cache cannot be trimmed safely, only the accepted tokens are advanced on the live cache. The rejected tail is never allowed to pollute the future prefix state.

4.4 Deciding When to Use the Drafter

Drafting from local text is sometimes the wrong choice. Open-ended prompts often accept only a few draft tokens, so verification overhead dominates. MachBoost therefore exposes a benchmark step. For a prompt family, it measures:

- token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can then calibrate an accelerator:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If the measured workload fails the policy, `generate` uses the serial baseline path. This is not as exciting as always-on acceleration, but it is much safer for a package people may drop into arbitrary local workflows.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. A service with only `next_token` can still run through the wrapper, but it cannot skip target work. A service with `verify` can accept several draft tokens per target call. The high-level `Accelerator` API loads MLX or Hugging Face models, reads strings, files, or directories as context, and exposes `benchmark`, `calibrate`, and `generate`. The Ollama HTTP adapter is intentionally limited: the public API does not expose token IDs, logits, KV-cache snapshots, or verifier hooks, so it is useful for benchmarking and option management but not for this form of acceleration.

6 Evaluation

6.1 Setup

We evaluate on a Mac with Apple M1 Max, 10 CPU cores, 32 GB unified memory, and macOS 26.5.2. The Python environment includes MLX 0.31.2 and `mlx-lm` 0.31.3. The evaluated model is `mlx-community/Qwen3.5-0.8B-MLX-4bit`. The initial suite comes from `results/mlx_evidence_suite_20260701.json`; the longer strict-mode suite comes from `results/mlx_evidence_v2_strict_c1_20260706.json`.

The suite contains seven fixture families with five fresh nonce-bearing repeats each: policy continuation, structured JSON continuation, retrieval-style answering, code completion, repository command quote, creative open-ended generation, and short answer. Each run generates up to

Metric	Value
Runs	35
Output match rate	100%
Median baseline speed	135.02 tok/s
Median boosted speed	188.13 tok/s
Median speedup	1.38×
Mean speedup	1.32×
P90 speedup	1.62×
Median accepted draft tokens	23.0
Median forward reduction	70.83%

Table 1: Overall MLX evidence-suite results. Output match compares token IDs between serial greedy baseline and boosted generation.

24 tokens. The positive fixtures are designed to have real local-context overlap; the creative and short-answer fixtures are negative controls.

We also run a stricter 64-token suite with three additional real-artifact fixtures drawn from the repository README, the core Python implementation, and this paper source. That run disables the MLX prompt cache for both baseline and boosted paths, and drafts only from provided context rather than from the full prompt wrapper. It is slower in absolute tokens per second than cache-enabled decoding, but it avoids cache-trajectory drift and gives a cleaner exactness check for longer generations.

Two smaller comparison runs are included as checks rather than as final evidence. The first compares serial Hugging Face generation, Hugging Face prompt lookup, and MachBoost on `Qwen/Qwen2.5-3B-Instruct`. The second is a two-fixture smoke run on `mlx-community/Qwen3.5-9B-MLX-4bit`. Their artifacts are `results/hf_prompt_lookup_compare_qwen25_3b.json` and `results/mlx_qwen35_9b_strict_smoke.json`.

6.2 Overall Results

Across the full suite, the boosted path matches the serial baseline in every run. Median throughput increases from 135.02 to 188.13 tokens per second, corresponding to a 1.38× median speedup. The p90 speedup is 1.62×.

6.3 Per-Workflow Results

The per-fixture view is more informative than the aggregate number. Policy, JSON, and code continuation all exceed 1.60× median speedup. Repository quote is weaker but still useful at 1.38×. Retrieval-style answering improves only 1.12×; the local text helps, but fewer tokens are accepted per verification call. The negative controls are a useful sanity check: they still match the baseline, but they are slower. Their continuations are not mostly recoverable from the provided context, so the extra verifier calls are wasted.

6.4 Strict 64-Token Run

The 64-token run is useful for a different reason than the first suite. It is not the fastest way to serve MLX models, but it shows that longer grounded spans can be accepted cleanly when the verifier and baseline use the same stateless execution mode. The three real-artifact fixtures are particularly

Fixture	Expectation	Match	Speedup	Base tok/s	Boost tok/s
Policy quote	Positive	100%	1.62×	125.84	203.78
JSON continuation	Positive	100%	1.61×	125.55	202.66
Code completion	Positive	100%	1.60×	125.13	201.84
Repo quote	Positive	100%	1.38×	136.52	188.13
RAG answer	Positive	100%	1.12×	135.02	151.57
Creative open-ended	Negative	100%	0.95×	157.63	148.91
Short answer	Negative	100%	0.90×	156.42	140.89

Table 2: Median per-fixture results over five repeats. Positive fixtures contain expected local-context overlap; negative fixtures are included to test gating.

Fixture	Match	Speedup	Accepted
Real README continuation	100%	5.68×	64
Real core-code continuation	100%	5.93×	64
Real paper continuation	100%	8.54×	64
JSON continuation	100%	3.79×	57
Code completion	100%	3.72×	55
Policy quote	100%	2.31×	41
RAG answer	100%	1.91×	27
Repo quote	100%	1.78×	24
Creative open-ended	100%	1.09×	0
Short answer	100%	1.01×	0

Table 3: Median results for the 64-token strict MLX run over three repeats. Prompt cache is disabled for both baseline and boosted paths, and draft source is limited to context text. Overall median speedup is 3.03× over 30 runs.

strong: all accept the full 64-token budget and exceed 5.6× median speedup. In contrast, the two negative controls accept no draft tokens and stay near 1.0×.

6.5 Comparison with Hugging Face Prompt Lookup

This comparison changes the interpretation. Hugging Face prompt lookup is already a strong baseline. With a 16-token lookup budget it reaches a 2.11× median speedup and reduces median forward calls from 32 to 3. MachBoost context mode is faster on the median fixture mix at 2.47×, but it does not dominate every fixture. Hugging Face prompt lookup is stronger on JSON, RAG, and code in this run; MachBoost context is stronger on the local README, policy, and slightly on core-code continuation. Four of the six Hugging Face prompt-lookup rows emitted extra raw tail tokens after the matching 32-token prefix, so the artifact records both fixed-budget token IDs and raw generated length.

6.6 Larger-Model Smoke Run

The 9B result is deliberately labeled a smoke run. It is too small to support a broad claim, but it matches the expected scaling behavior: when the draft is accepted, avoiding serial target-model steps matters more as the target model gets slower.

Method	Match	Speedup	tok/s	Forwards
HF serial generate	100%	1.00×	21.74	32.0
HF prompt lookup 4	100%	1.83×	38.68	8.5
HF prompt lookup 8	100%	1.86×	41.13	5.5
HF prompt lookup 16	100%	2.11×	46.59	3.0
MachBoost prompt	100%	2.14×	45.77	10.5
MachBoost context	100%	2.47×	51.06	10.5
MachBoost prompt+context	100%	2.26×	47.63	11.0

Table 4: Direct Hugging Face comparison on `Qwen/Qwen2.5-3B-Instruct`, six fixtures, one repeat, and 32 requested new tokens. Match is token-prefix equality over the requested budget.

Model/fixture	Match	Speedup	Base tok/s	Boost tok/s
Qwen3.5-9B policy	100%	6.99×	1.86	13.00
Qwen3.5-9B JSON	100%	8.01×	1.69	13.51

Table 5: Two-row strict MLX smoke run on `mlx-community/Qwen3.5-9B-MLX-4bit`, 16 tokens. Median speedup is 7.50× with full draft acceptance.

6.7 Interpretation

These results are not evidence for a universal speedup layer. They instead suggest that grounded local workloads are separable from open-ended ones with a small amount of measurement. That distinction matters for developer tools. A repo assistant, config editor, policy search tool, or command helper can benchmark representative prompts once and avoid using the drafter where acceptance is low.

7 Limitations

The evidence here is still early. The strongest repeated benchmark uses one small MLX model on one Apple Silicon machine; the longer suite reaches 64 tokens but still uses controlled fixtures rather than broad real applications. The Hugging Face comparison has only one repeat, and the 9B result is only a two-row smoke run. The method still needs repeated 3B/8B/9B evaluations, sampled-decoding tests, and larger real repositories. The drafter is also deliberately simple; unlike PLD+, it does not rank spans with model artifacts. Finally, the cache-enabled MLX path is faster in raw throughput but can diverge on long boundary cases, so the strict suite disables the prompt cache for evidence runs. Ollama over HTTP remains outside the accelerated path unless the runtime exposes a native verifier hook or a patched runner.

There is also a methodological limit. Token equality is a strong check for greedy decoding, but it says little about semantic quality under sampling. For that reason, this paper makes no quality claim. The goal is lower latency when the desired output is the same.

8 Future Work

Several next steps are straightforward. The evaluation needs repeated larger-model runs, longer generations, and real workspaces. The drafter could use suffix automata, prompt/document seg-

mentation, model-artifact ranking, or retrieval over multiple corpora. The decision rule could also become online: try the drafter for a short prefix, back off when acceptance is low, and re-enable it inside structured regions. Finally, a native Ollama or llama.cpp verifier hook would make the approach available to many more local users.

9 Conclusion

MachBoost is a practical experiment in using nearby text as the draft source for speculative decoding. It is not a shortcut for every prompt, and it is not the first prompt-lookup method. On the measured MLX suite, however, it matches serial greedy decoding across all 35 initial runs and reaches a 1.38× median speedup overall, with 1.60–1.62× speedups on the strongest grounded tasks. A stricter 64-token MLX run adds 90 exact-match rows across three runs and a 3.00× aggregate median speedup against stateless greedy decoding. The first direct Hugging Face comparison shows that built-in prompt lookup is strong, while MachBoost's local-context mode remains competitive and slightly faster on the median of the tested fixtures. The package design follows the main lesson from the results: use the drafter when local context predicts the next tokens, and fall back when it does not.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] Hugging Face. Assisted decoding. https://huggingface.co/docs/transformers/en/assisted_decoding, 2026. Accessed July 6, 2026.

[5] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[6] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[7] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[8] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical*

Methods in Natural Language Processing: System Demonstrations, pages 38–45. Association for Computational Linguistics, 2020.

[9] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[10] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.